

Pràctica 3

Jaume Adrover Fernández
43235882E

Evaluación del sistema actual

1. Explica con detalle cómo es el diseño y la implementación del experimento para evaluar el sistema actual. Se deben justificar las decisiones tomadas, desde el número de muestras que se van a tomar hasta qué monitores se van a lanzar y por qué.

El objetivo de esta práctica es el estudio del rendimiento del sistema actual. El experimento que vamos a diseñar para este consta de 4 fases:

1. Comprensión del sistema a evaluar
2. Caracterización de la carga actual del sistema
3. Recolección de datos y parametrización
4. Análisis de los resultados obtenidos

1.1 Comprensión del sistema a evaluar

Primero de todo, debemos conocer las especificaciones del sistema a evaluar. En concreto, nos interesa el número de CPUs y la cantidad de memoria principal, ya que ambos serán los protagonistas en este benchmark. Para determinar el número de CPUs vamos a utilizar el comando `lscpu`. Además, nos interesa también el número de hilos. Vamos a utilizar los siguientes comandos:

```
lscpu
```

El comando `lscpu` nos muestra todas las especificaciones de nuestro procesador, del cual vamos a enfocarnos en los más importantes como la frecuencia o el número de CPUs.

```
grep "processor" /proc/cpuinfo | sort -u | wc -l
```

Sin embargo, este nos indica el número de hilos totales por si hay confusión con el comando anterior. Para finalizar con la descripción de nuestro sistema, ejecutamos el comando `top` para saber la cantidad de memoria total en nuestro sistema:

```
top -b -d1 -n1 | grep -i "KiB Mem"
```

Característica	Datos
Modelo	11th Gen Intel(R) Core(TM) i5-11357G
Arquitectura	x86_64 (64 bits)
Número de CPUs	2
Frecuencia	2.40 GHz
Número de Hilos	2
Cantidad de memoria principal	2643976 KiB
Sistema Operativo	Ubuntu 16.04 (Linux)
Precio	899€

1.2 Caracterización de la carga actual del sistema

Conocidas ya nuestras especificaciones del sistema, vamos a caracterizar la carga actual del sistema. Vamos a realizar un benchmarking mediante el comando `sysbench` que nos proporciona Linux. En concreto, utilizamos la **versión 0.4.12**. Este sigue la estructura mostrada a continuación:

```
sysbench --test=cpu --cpu-max-prime=<> --num-threads<> run
```

El comando `sysbench` realiza la búsqueda de números primos dentro del intervalo introducido por consola. Los parámetros se indican con "`<>`", donde el primero corresponde al extremo superior del intervalo donde se realizará la búsqueda y el segundo para el número de hilos implicados en la ejecución. Vamos a ejecutar este comando para las cargas de 25.000, 50.000, 100.000, 150.000 y 250.000. Es decir, se buscará en intervalos de esa longitud los números primos con el propósito de darle trabajo a la CPU, que se encarga de cálculos aritmético-lógicos. Además, he añadido 250.000, ya que será un dato a utilizar en apartados posteriores.

Para obtener máxima precisión en las medias, se debería ejecutar un número razonable de veces para obtener suficientes muestras. En mi caso, he elegido 10 ejecuciones por carga, con el ya dicho objetivo de aumentar la precisión en los datos obtenidos. He realizado un script para poder ejecutar automáticamente esta tarea y no estar pendiente del ordenador cada vez que se termina una ejecución. Cabe

destacar que las condiciones del experimento son con un 100% de batería y enchufado, sin conexión a internet. Estas condiciones nos aseguran que se utiliza el porcentaje máximo de CPU y memoria ya que los portátiles reducen el % de rendimiento cuando no están conectados a la corriente. Además, desconectar el internet nos asegurará que no se realice ninguna descarga en segundo plano. Veamos el script:

```
#SCRIPT PARA REALIZAR BENCHMARKING
#Programa que para cada carga recoge 10 muestras y las almacena en el
fichero sysbench2.txt

#Bucle para cargas 25.000,50.000,100.000,150.000 y 250.000
for i in 25000 50000 100000 150000 250000
do
echo "EMPIEZA BENCHMARK DE CARGA $i"
#10 ejecuciones por carga
for j in 0 1 2 3 4 5 6 7 8 9
do
    sysbench --test=cpu --cpu-max-prime=$i --num-threads=1
run>>sysbench2.txt
echo $'\n\nFINALIZADO BENCHMARKING ' "$j" ' a las ' $(date) >>
sysbench2.txt
sleep 25
echo
$'_____ \n\n'>>sysbench2.t
xt

done
done
```

El script siguiente va iterando y notifica por pantalla cada vez que se empieza un benchmark de carga indicada por la variable 'i' que funciona como contador en el bucle for. Además, separa mediante una línea y dos saltos de línea (\n) cada muestra, con el objetivo de que se puedan separar y observar más fácilmente. Además, añadimos la hora de finalización en el fichero y realizamos un sleep. Esta instrucción es muy importante ya que nos ayudará a separar nuestras muestras. Tendremos 5 muestras (25 segundos del sleep divididos entre 5) que nos presentarán una inactividad muy superior a las demás, por lo que podremos afirmar que estas nos indican la finalización de una ejecución y el periodo de descanso hasta empezar otro benchmark, si no hemos terminado de ejecutar todo el script.

Por otro lado, nos interesa conocer cómo es el comportamiento de la CPU y el uso de memoria mientras se está ejecutando el script anterior. Para esta tarea, haremos uso de los monitores top y vmstat, aunque hay otras posibilidades. Hago

uso del monitor TOP para poder ver el comportamiento de la CPU ya que este incluye los datos de uso del modo usuario, sistema y global(mediante la inactiva).

Para ver la utilización de la memoria, utilizaré vmstat ya que aunque el comando TOP nos proporciona esa información, prefiero utilizar este monitor porque me es más fácil realizar cálculos mediante una sola columna, que nos proporciona VMSTAT mediante el filtro awk. Es decir, utilizo vmstat para poder solo obtener la columna de memoria libre y poder realizar cálculos sobre una única columna en lugar de utilizar el comando top, que nos proporciona más datos que no nos son útiles, como la memoria del buffer. Como en el método anterior, mediante los tiempos de descanso del comando sleep, se generarán unas muestras con unos valores elevados de inactividad que nos serán útiles para separar las muestras de cada ejecución y así ver cómo cambian según las cargas.

Para poder calcular el número de muestras que vamos a tomar, ejecutaremos el comando time para cada sysbench con cada respectiva carga. Así, multiplicaremos los tiempos de ejecución por 10, más sus respectivos sleeps para poder dividir este resultado en 5, obteniendo así el número de muestras. Si observamos que este número de muestras nos da decimales, redondeamos al siguiente entero.

CARGA (Números primos)	TIEMPO APROXIMADO DE EJECUCIÓN (segundos)	MUESTRAS POR EJECUCIÓN(T=5)
25.000	10	2
50.000	25	5
100.000	67	14
150.000	120	24
250.000	245	50

$$S_{25k} = 2 \text{ muestras}$$

$$S_{50k} = 5 \text{ muestras}$$

$$S_{100k} = 14 \text{ muestras}$$

$$S_{150k} = 24 \text{ muestras}$$

$$S_{250k} = 50 \text{ muestras}$$

$$N_{\text{muestras}} = \sum_{i=0}^n 10(S_i)$$

$$N_{muestras} = 10(S_{25k} + S_{50k} + S_{100k} + S_{150k} + S_{250k}) = 950 \text{ muestras}$$

Hemos calculado, por tanto, el número de muestras útiles para nuestro experimento. A estas cabe añadirles más muestras, en concreto 50 de los periodos de espera con sleep y otras 25 para que no nos falten muestras en caso de haber realizado los cálculos de manera errónea. Estas 25 últimas muestras funcionan como margen de error. Habiendo calculado el número total de muestras(1025 en nuestro caso), los comandos quedan así:

```
top -b -d5 -n1025 | grep -i "%Cpu(s):" >>top.txt
```

```
vmstat 5 1025 | awk '{print $4}' >> vmstat.txt
```

1.3 Recolección de datos y parametrización

Para la recolección de datos vamos a lanzar en orden los siguientes procesos:

- 1.Lanzar el monitor top en una terminal externa
- 2.Lanzar el monitor vmstat en una terminal externa
- 3.Ejecutar el script realizado

Estos datos se almacenan en los ficheros indicados anteriormente en los comandos, llamados **top.txt**, **vmstat.txt** y **sysbench.txt**.

Para poder trabajar y hacer cálculos sobre las muestras, debemos importar los datos en una hoja de cálculo, en nuestro caso es **Google Sheets**. Hechas las importaciones, tenemos todas las muestras en el excel y procedemos a manipular los datos de manera que nos sea más cómodo trabajar sobre ellos. Es decir, agrupamos las muestras por carga y número de ejecución. Después de esta labor, podemos eliminar las muestras que presentan un porcentaje elevado de inactividad mediante un filtro, ya que solo nos sirven para diferenciar los intervalos de datos.

El resultado de esto es una hoja de cálculo con tres hojas, una para el vmstat, top y para el **sysbench**. Cada una tiene los datos organizados de manera que ya se puede proceder al análisis.

1.4 Análisis de los resultados obtenidos

Para finalizar, escogemos qué tipo de gráficos vamos a utilizar que nos permitan representar mejor y poder sacar unas conclusiones acertadas en base al

estudio. Las principales relaciones que vamos a mostrar son las siguientes: UsoCPU-Tiempo, UsoMem-Tiempo y TiempoRespuesta-Cargas.

1.4.1 Análisis UsoCPU-Tiempo

En el análisis de la siguiente relación observaremos cómo se comporta el uso de la CPU a medida que va pasando el tiempo de ejecución. Por tanto, crearemos un gráfico donde el eje abscisas(X) queda determinado por el tiempo y el eje de las ordenadas(Y) refleja el porcentaje de uso de la CPU. Los intervalos de los ejes vienen descritos de la siguiente manera:

$$Eje_x = x \in \{0, 4750\}$$

$$Eje_y = f(x) \in \{0, 100\}$$

Las unidades a utilizar son segundos en el eje X y porcentaje sobre 100(%) en el eje Y. Además, escogemos un gráfico de tipo áreas para reflejar mejor estos datos.

1.4.2 Análisis UsoMemoria-Tiempo

En el análisis de esta relación veremos la evolución del porcentaje que usa la memoria a medida del tiempo de ejecución. El eje de las abscisas demuestra el tiempo de ejecución y las ordenadas el porcentaje de uso de la memoria principal. Veamos los intervalos:

$$Eje_x = x \in \{0, 4750\}$$

$$Eje_y = f(x) \in \{0, 100\}$$

Las unidades a utilizar son segundos en el eje X y porcentaje sobre 100(%) en el eje Y. El tipo de gráfico será de áreas.

1.4.3 Análisis TiempoRespuesta-Cargas

Y por último, observaremos cuanto tardan los procesos a ejecutar mediante el comando sysbench a medida que vamos cambiando las cargas. El eje de las Y no tiene un máximo ya que su tiempo depende del número de cargas, es decir, el eje X.

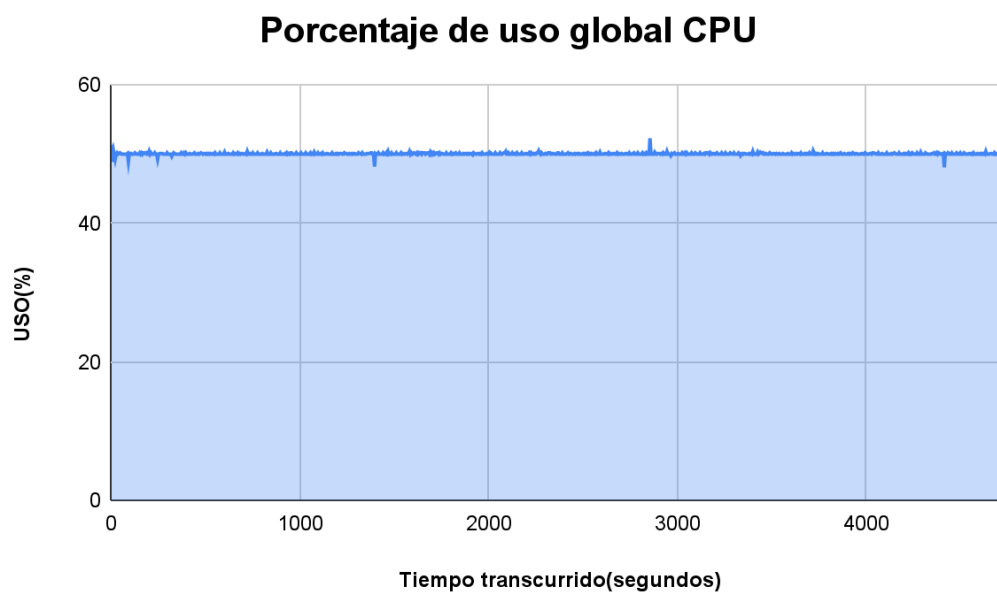
$$Eje_x = x \in \{25.000, 50.000, 100.000, 150.000, 250.000\}$$

$$Eje_y = f(x) \in \{0, \infty\}$$

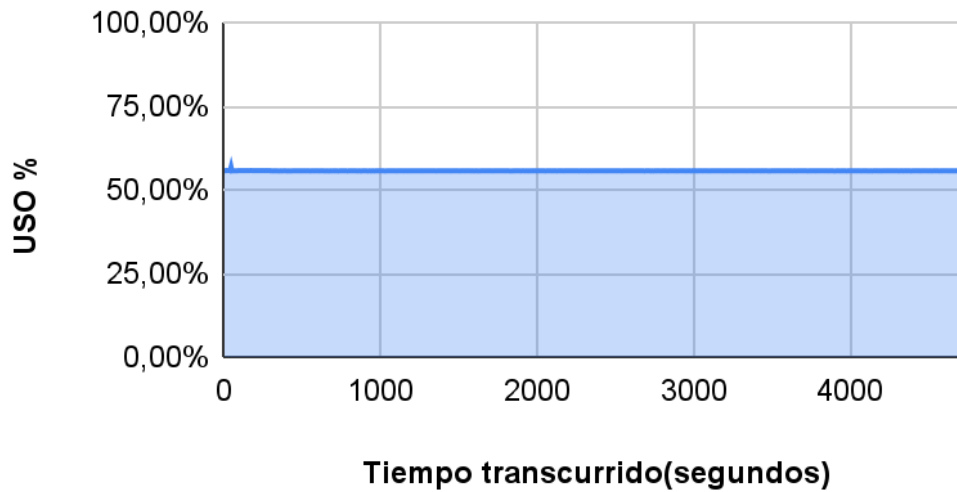
Las unidades de cada eje son números para el eje X, y segundos para el eje Y. Como en los casos anteriores, utilizaremos un gráfico del tipo área para poder ilustrar mejor los datos obtenidos.

2. ¿Cómo se comporta el sistema actual si variamos la carga varía en 25000, 50000, 100000 y 150000 números primos? ¿Cómo es el comportamiento del tiempo de respuesta y la productividad? Indica el valor para cada una de las ejecuciones del experimento y la media entre todas ellas.

Para poder observar el comportamiento del sistema actual respecto las cargas debemos realizar unas gráficas que nos muestren la tendencia que tienen el uso (%) de la CPU y de la memoria. Vamos a mostrar estas junto al tiempo total de ejecución, sin diferenciar intervalos, para poder ver su evolución más fácilmente en un solo gráfico.



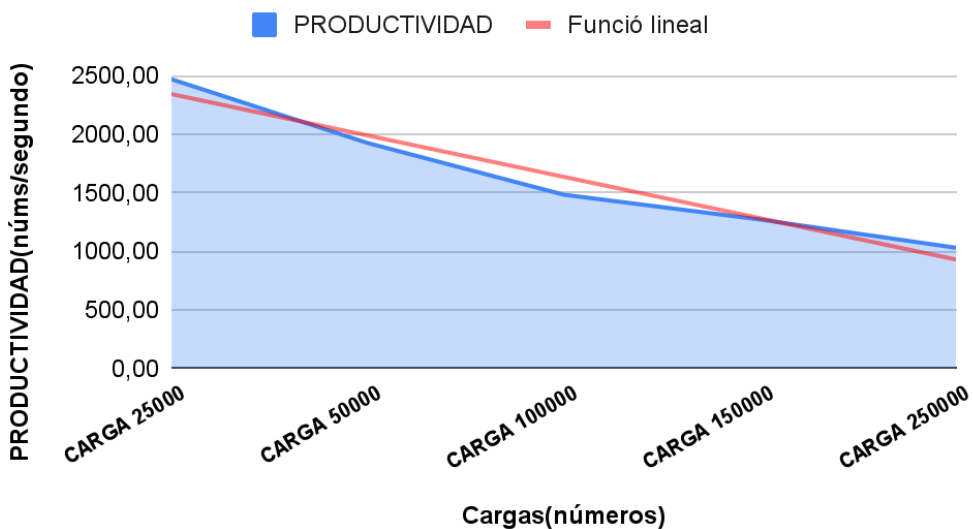
Porcentaje uso de la memoria



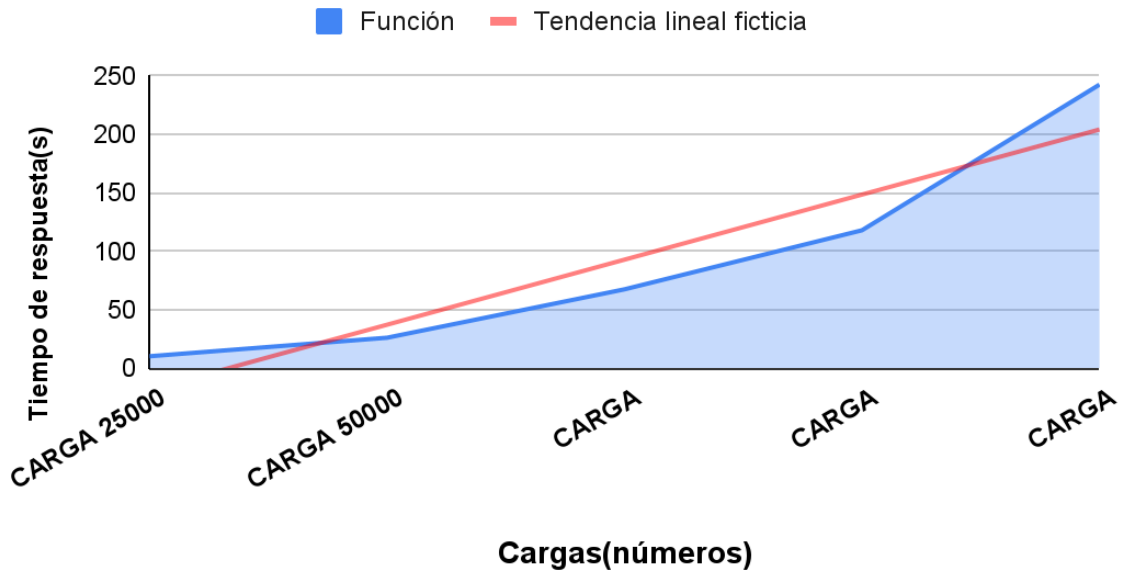
Como hemos observado a continuación, vemos que el sistema tiene una tendencia a utilizar solo el 50% de la CPU, ya que hemos especificado la mitad de hilos del procesador en el comando sysbench. Sin embargo, la memoria no sigue este patrón, sino que sigue intacta y no experimenta cambio alguno. Esto es debido a que la memoria no es utilizada en absoluto por el trabajo de las cargas y permanece estable. El número de cargas no influye en el uso de la CPU ni la memoria ya que no experimentamos ni aumento ni disminución alguna.

A diferencia de la primera parte, la productividad y los tiempos de respuesta si que sufren un comportamiento, que se puede ver a simple vista con los gráficos insertados a continuación:

PRODUCTIVIDAD MEDIA POR CARGAS



Función tiempos de respuesta por carga



En ambos, hemos añadido una función lineal ficticia para poder comparar y ver que no siguen esta distribución, sino que presentan un comportamiento no linealmente dependiente de la carga. Por ejemplo, cuando pasamos de 25.000 números primos a 50.000, el tiempo no experimenta un incremento 2x(doble), sino que pasamos de 10 a 25. Lo mismo pasa de 50.000 a 100.00, pasamos de un tiempo medio de ejecución de 25,97 hasta 67,36. Si calculamos la proporción entre tiempos de carga, veremos que los resultados no son los mismos:

$$T_i = \{k \cdot T_j \mid (j < i) \wedge (i, j \in N)\}$$

$$k_1 = \frac{T_{50k}}{T_{25K}} = \frac{25,97}{10,13} = 2.566$$

$$T_{50K} = 2.566 \cdot T_{25k}$$

$$k_2 = \frac{T_{100k}}{T_{50K}} = \frac{67,36}{25,97} = 2.593$$

$$T_{100k} = 2.593 \cdot T_{50K}$$

$$k_3 = \frac{T_{150k}}{T_{100K}} = \frac{117,84}{67,36} = 1.749$$

$$T_{150K} = 1.749 \cdot T_{100K}$$

$$k_4 = \frac{T_{250k}}{T_{150K}} = \frac{242,36}{117,84} = 2.056$$

$$T_{250K} = 2.056 \cdot T_{150K}$$

Siendo **k** el número a multiplicar por el tiempo anterior obtenido(j<i), podemos ver que este cambia y no es constante en la obtención de cada valor. Si este fuera igual en todas las anteriores ecuaciones, podríamos afirmar que esta función es linealmente dependiente de la carga. La carga implica un aumento del tiempo pero no de manera uniforme y lineal.

En cuanto a la productividad, vemos que disminuye en función de la carga pero tampoco de manera lineal. Puede apreciarse comparando con la línea ficticia o observando detenidamente el gráfico. Si queremos comprobar mediante el método anterior las proporciones entre estos, definimos una ecuación que nos permite obtener una constante K, similar al procedimiento anterior.

$$P_i = \{k \cdot P_j \mid \exists i, j \in N \wedge (j < i)\}$$

$$\begin{aligned} k_1 &= \frac{P_{50k}}{P_{25K}} = \frac{1925.47}{2469.74} = 0.78 & P_{50K} &= 0.78 \cdot P_{25k} \\ k_2 &= \frac{P_{100k}}{P_{50K}} = \frac{1484.50}{1925.47} = 0.78 & P_{100k} &= 0.78 \cdot P_{50K} \\ k_3 &= \frac{P_{150k}}{P_{100K}} = \frac{1272.9}{1484.50} = 0.8574 & P_{150K} &= 0.8574 \cdot P_{100K} \\ k_4 &= \frac{P_{250k}}{P_{150K}} = \frac{1031.54}{1272.9} = 0.81 & P_{250K} &= 0.81 \cdot P_{150K} \end{aligned}$$

Aunque no parece lineal, podemos observar que la relación entre ambos no cambia tanto como en el anterior caso, por lo que podemos confirmar que sigue una distribución más cercana a la lineal, aproximándose a esta.

Las fórmulas utilizadas en este apartado son las siguientes:

$$Productividad = \frac{Trabajos}{Unidad\ de\ tiempo} = \frac{Números}{Segundos}$$

$$TiempoMedio_{RESP} = \frac{1}{n} \sum_{i=0}^n t_i$$

Adjuntamos los datos de cada carga con sus respectivas iteraciones utilizados para los cálculos anteriores(hemos utilizado las medias):

	CARGA	TOTAL TIME	PRODUCTIVIDAD	USO CPU
Muestra 1	25000	10,05	2487,44	50%
Muestra 2	25000	10,04	2490,91	50%
Muestra 3	25000	10,01	2496,38	50%
Muestra 4	25000	10,66	2344,89	50%
Muestra 5	25000	10,38	2407,90	50%
Muestra 6	25000	9,97	2506,44	50%
Muestra 7	25000	10,00	2498,98	50%
Muestra 8	25000	10,11	2473,53	50%
Muestra 9	25000	10,03	2493,47	50%
Muestra 10	25000	10,01	2497,43	50%
MEDIA	X	10,13	2469,74	X

	CARGA	TOTAL TIME	PRODUCTIVITY	USO CPU
Muestra 1	50000	25,91	1929,76	50%
Muestra 2	50000	26,23	1906,21	50%
Muestra 3	50000	25,92	1929,01	50%
Muestra 4	50000	25,91	1929,76	50%
Muestra 5	50000	25,90	1930,50	50%
Muestra 6	50000	26,17	1910,58	50%
Muestra 7	50000	25,99	1923,59	50%
Muestra 8	50000	25,9	1930,50	50%
Muestra 9	50000	25,87	1932,74	50%
Muestra 10	50000	25,88	1931,99	50%
MEDIA	X	25,97	1925,47	X

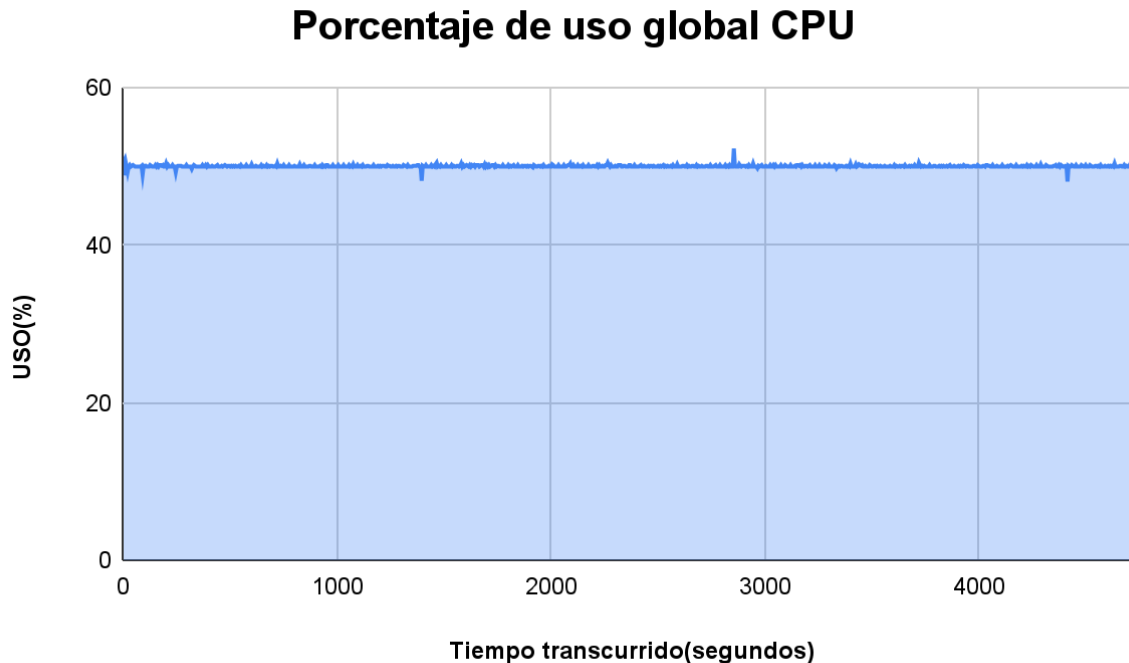
	CARGA	TOTAL TIME	PRODUCTIVIDAD	USO CPU
Muestra 1	100000	67,43	1483,02	50%
Muestra 2	100000	67,54	1480,60	50%
Muestra 3	100000	67,34	1485,00	50%
Muestra 4	100000	67,35	1484,78	50%
Muestra 5	100000	67,29	1486,10	50%
Muestra 6	100000	67,29	1486,10	50%
Muestra 7	100000	67,54	1480,60	50%
Muestra 8	100000	67,26	1486,77	50%
Muestra 9	100000	67,24	1487,21	50%
Muestra 10	100000	67,35	1484,78	50%
MEDIA	X	67,36	1484,50	X

	CARGA	TOTAL TIME	PRODUCTIVIDAD	USO CPU
Muestra 1	150000	117,80	1273,34	50%
Muestra 2	150000	117,80	1273,34	50%
Muestra 3	150000	117,67	1274,75	50%
Muestra 4	150000	117,88	1272,48	50%
Muestra 5	150000	117,89	1272,37	50%
Muestra 6	150000	117,89	1272,37	50%
Muestra 7	150000	117,83	1273,02	50%
Muestra 8	150000	117,80	1273,34	50%
Muestra 9	150000	117,80	1273,34	50%
Muestra 10	150000	118,05	1270,65	50%
MEDIA	X	117,84	1272,90	X

	CARGA	TOTAL TIME	PRODUCTIVIDAD	USO CPU
Muestra 1	250000	241,53	1035,07	50%
Muestra 2	250000	241,73	1034,21	50%
Muestra 3	250000	243,52	1026,61	50%
Muestra 4	250000	242,88	1029,31	50%
Muestra 5	250000	242,01	1033,02	50%
Muestra 6	250000	241,33	1035,93	50%
Muestra 7	250000	243,07	1028,51	50%
Muestra 8	250000	242,37	1031,48	50%
Muestra 9	250000	243,09	1028,43	50%
Muestra 10	250000	242,04	1032,89	50%
MEDIA	X	242,36	1031,54	X

3. ¿Cuál es el porcentaje de CPU y de memoria del sistema para cada una de las cargas ejecutadas? ¿Por qué se produce ese comportamiento? Muéstralo gráficamente a lo largo del tiempo de ejecución de la carga

Para hacer un análisis del porcentaje de CPU obtenido con el comando sysbench, vamos a graficar los resultados para que se vea de una manera más clara:



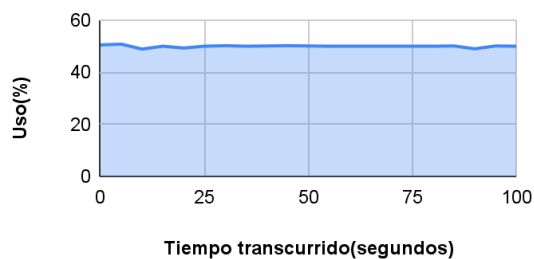
El primer gráfico que podemos observar es el porcentaje de CPU, con el total de tiempo transcurrido, es decir, incluyendo todas las cargas con sus respectivas iteraciones. Como se puede observar, vemos que el uso es muy estable, acercándose al 50% sin variaciones muy significativas. Esto es debido a que hemos especificado la carga de trabajo sobre la CPU al 50%, por lo que el trabajo siempre consume la misma cantidad de CPU. En concreto, utilizamos la mitad de hilos proporcionados por el procesador.

Veamos una tabla que nos separa por intervalos y su uso medio:

CARGA	INTERVALO(segundos)	USO MEDIO CPU (%)
25.000	0-100	49.96%
50.000	100-350	50.04%
100.000	350-1050	50.05%
150.000	1050-2250	50.06%
250.000	2250-4750	50.06%
DESVIACIÓN TÍPICA	X	0.04%

Como ya hemos dicho anteriormente, el uso medio de la CPU tiende al 50% debido a que nosotros hemos especificado ese parámetro en el comando sysbench. Si volvemos a observar las cargas por individual, apreciamos que no hay variaciones y siempre sigue el mismo patrón, sea la carga que sea. El número de números primos a buscar no afecta ni influye en un aumento de porcentaje. Nosotros asignamos que % queremos que se use.

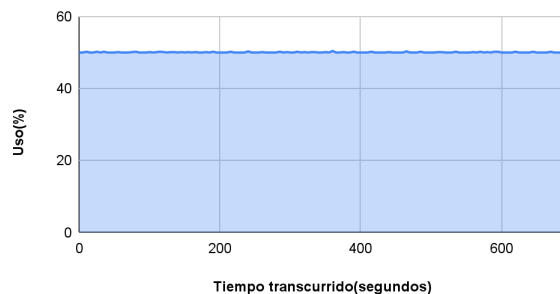
Porcentaje uso CPU carga 25k



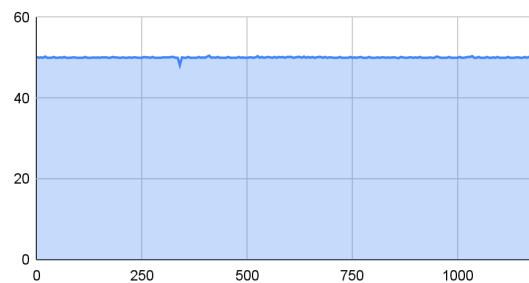
Porcentaje uso global CPU carga 50k



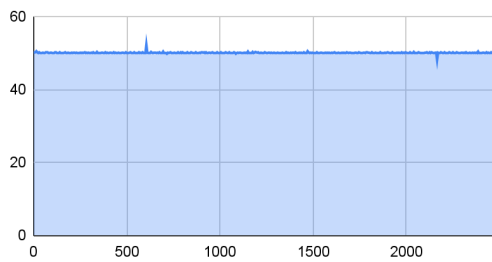
Porcentaje uso global CPU carga 100k



Porcentaje uso global CPU carga 150k

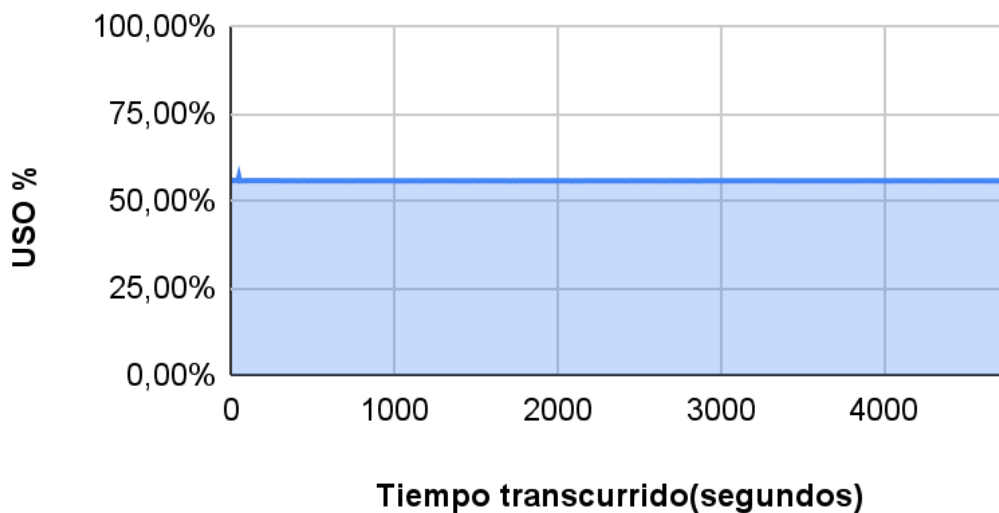


Porcentaje uso global CPU carga 250k



Por otro lado, también nos interesa ver el porcentaje de uso de la memoria. Como podemos observar a continuación, la memoria tiene un comportamiento peculiar.

Porcentaje uso de la memoria



CARGA	INTERVALO(s)	USO MEDIO MEMORIA(%)	Capacidad media utilizada (KiB)
25.000	0-100	55.82%	1475994,095
50.000	100-350	55.78%	1474815,32
100.000	350-1050	55.73%	1473423,679
150.000	1050-2250	55.73%	1473412,692
250.000	2250-4750	55.73%	1473431,734
DESVIACIÓN TÍPICA	X	0,04%	1152,523048

Como podemos apreciar, observamos que el uso de la memoria no sufre cambio apenas. El motivo por este comportamiento se debe a que esta no es usada en operaciones aritmético-lógicas, por lo que el trabajo es realizado exclusivamente por la CPU.

En conclusión, con el análisis realizado, podemos apreciar que el trabajo es realizado principalmente por el procesador, utilizando constantemente la mitad de sus hilos. Este hecho se refleja en el porcentaje de uso de esta, que sigue una línea recta con el valor aproximado de un 50%. Este valor casi no cambia, teniendo las muestras una desviación típica muy baja, de un 0.04%. La desviación típica nos indica cuánto se alejan las muestras de la media, por lo que podemos afirmar que las muestras son consistentes y no padecen de muchos altibajos, resultado que no tendría sentido alguno en este contexto. Además, añadimos que la memoria también tiene un uso constante ya que solo se utiliza para el funcionamiento del sistema, sin contribuir al trabajo del procesador.

Comparar el sistema actual con los servidores A y B.

1. Explica detalladamente cómo se va a realizar el proceso de benchmarking. Se deberán justificar todas las decisiones que se han tomado al respecto.

El proceso que vamos a realizar de benchmarking tiene el objetivo de comparar nuestro sistema actual con los servidores A y B. Así, calcularemos medidas de rendimiento como la aceleración o el rendimiento coste, comparando valores y diciendo cual nos aporta mayores prestaciones.

En primer lugar, vamos a comprender el sistema a evaluar, enfocándonos en los componentes principales como la CPU y la memoria principal. Este paso ya se ha realizado en el apartado anterior, por lo que adjunto la tabla de nuestro sistema actual, ya que los datos de los servidores A y B nos los proporciona el enunciado. La explicación está más detallada en la primera parte.

Característica	Datos
Modelo	11th Gen Intel(R) Core(™) i5-11357G
Arquitectura	x86_64 (64 bits)
Número de CPUs	2
Frecuencia	2.40 GHz
Número de Hilos	2
Cantidad de memoria principal	2643976 KiB
Sistema Operativo	Ubuntu 16.04 (Linux)
Precio	899€

A continuación, caracterizamos la carga actual del sistema, que consistirá en realizar unas pruebas similares a las de los otros servidores, ya que no tendría sentido comparar sistemas con una metodología de benchmarking diferente. El servidor A obtiene los tiempos mediante el comando sysbench, con una carga de 250.000 números, usando al 50% la CPU. Mientras tanto, el servidor B, ejecuta el stress-ng con una carga de 300.000 y un uso del 100% de la CPU. Hemos creado otro script para automatizar esta tarea y que se almacene en un fichero llamado stress.txt. Aquí el script stress.sh:

```
#SCRIPT PARA REALIZAR BENCHMARKING
#Programa que hace 10 iteraciones con carga 300.000 y un uso de un 100%
de la CPU

echo "EMPIEZA BENCHMARK!"
#10 ejecuciones
for i in 0 1 2 3 4 5 6 7 8 9
do
    stress-ng --cpu=2 --cpu-ops=300.000>>stress.txt
    echo '$\n\nFINALIZADO BENCHMARKING ' "$i" 'a las ' $(date) >>
    stress.txt
    sleep 25
    echo
    $'_____ \n\n'>>stress.txt
done
```

Este script, igual al anterior, realiza 10 iteraciones y va almacenando los datos en un fichero, junto a su duración y la fecha de finalización. De esta manera, nos es más

fácil la recolección de datos para no tener que ejecutar 10 comandos nosotros desde la terminal.

El comando stress-ng se encarga de realizar ciertas operaciones llamadas “stressors”. Este tiene dos parámetros a completar, el número de CPU’s y el número de operaciones a realizar. En nuestro caso cpu=2 y las operaciones, cpu-ops=300.000.

```
stress-ng --cpu <> --cpu-ops=<>
```

Finalmente, igual que en el procedimiento anterior, importamos el fichero en un excel y seleccionamos los datos que nos interesan. A continuación, realizaremos todos los cálculos necesarios y gráficos para poder representar la información de la manera más visual y cómoda, creando una tabla para guardar los tiempos de ejecución. El resultado es un excel con la información importante, con gráficos y cálculos de medias.

2. Considerando sólo el rendimiento de los servidores, ¿cambiarías el sistema actual por el servidor A? ¿Y por el B? ¿Por qué?

Para poder comparar el rendimiento de cada servidor y poder compararlos debemos escoger una unidad de rendimiento que sea común en ambos, obtenido en las mismas condiciones. En nuestro caso, tenemos los tiempos de ejecución de las respectivas cargas por servidor y obtenemos los siguientes datos para el sistema actual:

	Tiempo de ejecución(s)	Tiempo de ejecución(s)
Configuración	Carga=250.000 50% Cpu	Carga=300.000 100% Cpu
Muestra 1	241,53	452,34
Muestra 2	241,73	458,86
Muestra 3	243,52	453,67
Muestra 4	242,88	453,59
Muestra 5	242,01	452,96

Muestra 6	241,33	454,01
Muestra 7	243,07	453,48
Muestra 8	242,37	453,85
Muestra 9	243,09	453,27
Muestra 10	242,04	454,83
MEDIA	242,36	454,086

A continuación, escogemos una medida de rendimiento adecuada para nuestra comparación, en nuestro caso será la aceleración. Primero seleccionamos los tiempos para comparar adecuados. Para comparar, seleccionamos el tiempo medio de ejecución T_{s_1} , que trabaja sobre 250.000 con un uso de la CPU al 50%, usando el sysbench. Después, elegimos T_{s_2} , que realiza un stress-ng con 300.000 con un uso de la CPU al 100%.

$$T_{s_1} = 242,36$$

$$T_{s_2} = 454,086$$

$$T_A = 23,584$$

$$T_B = 26,998$$

$$Aceleración = \frac{T_{s_1}}{T_A} = \frac{242,36}{23,584} = 10,276 \rightarrow \mathbf{927.6\%}$$

$$Aceleración = \frac{T_{s_2}}{T_B} = \frac{454,086}{26,998} = 16,82 \rightarrow \mathbf{1582\%}$$

En cuanto a la aceleración del servidor A respecto al sistema actual, podemos obtener que el primero es aproximadamente 10,3 veces más rápido, o un 927,6%. Por otro lado, obtenemos que el servidor B es un 1582% más rápido, o un equivalente a unas 17 veces más rápido.

En conclusión, podemos observar que ambos servidores mejoran en gran cantidad el rendimiento de nuestro sistema actual, por lo que cualquier de los dos cambios sería una buena decisión en nuestro caso. Cabe destacar que no podemos decir que una opción sea mejor que otra ya que se han realizado con pruebas de referenciación diferentes, usando uno solo el 50% y el otro un 100%. Además, el sysbench y el stress-ng no funcionan de la misma manera.

3. Si además consideramos el coste económico, ¿cambiarías el sistema actual por el servidor A? ¿Y por el B? ¿Por qué?

Para considerar el coste económico en la comparación actual, utilizaremos el índice Rendimiento/Coste, véase la fórmula posterior. Esta necesita dos variables, el tiempo y el coste, ya que la primera es un claro índice de rendimiento y debe incluirse también. Cabe destacar que, realizadas dos pruebas diferentes sobre nuestro sistema actual, debemos calcular dos tasas diferentes, para poder relacionar cada una con el servidor correspondiente. Es decir, si tenemos el tiempo del comando sysbench con 250.000 y el de 300.000 con stress-ng, no podremos utilizar solo uno ya que el sysbench ha sido ejecutado por el servidor A y el stress-ng por el servidor B. No tendría sentido comparar el sistema actual con una carga diferente a la previamente utilizada.

Declaraciones de variables necesitadas:

-Tiempos

$$T_{ACTUAL_1} = 242.36s$$

$$T_{ACTUAL_2} = 454,086$$

$$T_A = 23.584 s$$

$$T_B = 26.998 s$$

-Costes

$$C_A = 1245€$$

$$C_B = 907€$$

$$C_{ACTUAL} = 899€$$

Fórmula general del rendimiento-coste:

$$RendimientoCoste = \frac{1}{Tiempo \cdot Coste}$$

Este índice nos relaciona las prestaciones con el coste, pudiendo así comparar no solo mediante su rendimiento. Como es obvio, buscamos maximizar este valor.

Cálculos:

$$RendimientoCoste_{ACTUAL_1} = \frac{1}{242.36.899} = 4.589 \cdot 10^{-6}$$

$$RendimientoCoste_{ACTUAL_2} = \frac{1}{454,086.899} = 2.45 \cdot 10^{-6}$$

$$RendimientoCoste_A = \frac{1}{23.584.1245} = 34.056 \cdot 10^{-6}$$

$$RendimientoCoste_B = \frac{1}{26.998.907} = 40.83 \cdot 10^{-6}$$

Realizados todos los cálculos, vamos a proceder a separar dos casos, la posibilidad de cambio para el servidor A y otra para el servidor B. Cabe aclarar que he calculado dos índices para el sistema actual ya que no tendría sentido comparar diferentes dispositivos mediante pruebas distintas.

- **CAMBIO SISTEMA ACTUAL POR SERVIDOR A**

$$RendimientoCoste_A > RendimientoCoste_{ACTUAL1}$$

$$34.056 \cdot 10^{-6} > 4.589 \cdot 10^{-6}$$

Como podemos apreciar, observamos que la tasa de rendimiento es mucho mayor para el servidor A que para el actual. Por este motivo, sí que nos convendría cambiar el sistema actual por el A.

- **CAMBIO SISTEMA ACTUAL POR SERVIDOR B**

$$RendimientoCoste_B > RendimientoCoste_{ACTUAL2}$$

$$40.83 \cdot 10^{-6} > 2.45 \cdot 10^{-6}$$

A continuación, observamos que el servidor B tiene un índice mucho mayor que el sistema actual, obtenido en igualdad de condiciones. Este hecho refleja que el servidor B es mejor en cuanto a proporción Rendimiento/Coste que nuestro actual. Es por este motivo que sí realizaríamos un posible cambio de sistema en estas condiciones

En conclusión, podemos observar que nos sería rentable hacer el cambio en ambos casos, ¿aunque cuál de los dos sería mejor? Al no haber realizado el benchmark en igualdad de condiciones en todas las comparaciones, no podemos elegir ninguno de

los dos ya que el servidor A solo utiliza un 50% de CPU y realiza un benchmarking diferente al servidor B, que utiliza un 100% de CPU. El primero utiliza sysbench, mientras que el segundo usa stress-ng. Sin embargo, cualquier cambio nos beneficiaría y sería adecuado en nuestro caso.

4. ¿Cómo influye la carga ejecutada en los servidores A y B en la decisión de cambio?

La carga ejecutada por cada servidor influye en la manera de poder comparar con nuestro sistema actual. Pongamos de ejemplo el servidor A, este realiza un sysbench de un 50% de CPU con 300.000 números primos. Para poder elegir si debemos realizar el cambio tenemos que ejecutar el mismo comando con el mismo porcentaje de CPU y números. En esta situación, somos capaces de comparar mediante algún índice de rendimiento y evaluar la cuantía de nuestra mejora para ver si es rentable el cambio. Lo mismo ocurre con el servidor B, tendríamos que realizar un stress-ng con el 100% de CPU y 300.000 operaciones. Al obtener los datos en estas mismas situaciones seríamos capaces de poder comparar.

Finalmente, cabe destacar que no se pueden comparar los 3 sistemas a la vez ya que el servidor A y B tienen muestras recogidas en diferentes situaciones y contextos. Solo existe la posibilidad de comparar individualmente nuestro sistema actual con el servidor A/B por separado, mediante la obtención de muestras en iguales condiciones. Para poder escoger cuál de los 3 sería mejor tendríamos que establecer unas condiciones y benchmarking idénticos en los tres sistemas.